

Lezione 22 parte 3 - Dependency Injection

Inversione di Controllo e Dependency Injection

Fino ad ora abbiamo usato Spring in modo quasi "ingenuo":

- **abbiamo annotato le classi con `@RestController` e `@Service`, e Spring le ha istanziate e gestite automaticamente.**

Ma non abbiamo mai spiegato il **meccanismo** che sta dietro a tutto questo. È arrivato il momento di farlo.

Guardando il codice dell'esercizio sugli utenti, noti qualcosa di strano: nel Controller istanziamo il Service così:

java



Java

```
1 private UtenteService service = new UtenteService();
```

E nel Service istanziamo il DAO così:

java



Java

```
1 private DAOUtenteMappa dao = new DAOUtenteMappa();
```

Stiamo usando `new` — stiamo creando noi gli oggetti.

Questo funziona, ma è il modo sbagliato di farlo con Spring.

Il problema è che stiamo creando dipendenze esplicite tra i layer:

- **il Controller sa che esiste `UtenteService`, il Service sa che esiste `DAOUtenteMappa`.**
- Se un domani volessimo cambiare il DAO con uno che usa un database reale, dovremmo modificare il Service.
Questo viola il disaccoppiamento che abbiamo costruito con tanta cura.

La soluzione è affidarsi a due principi fondamentali di Spring:

1. Inversione di Controllo

2. Dependency Injection.

Inversione di controllo

Una delle caratteristiche più famose di Spring è la sua implementazione del principio **IoC** (Inversion of Control).

È un pattern per cui un componente di livello applicativo riceve il controllo da un componente appartenente a una libreria riusabile — solitamente un framework.

Per capirlo vale la pena confrontare i tre livelli di "attività":

1. La libreria (es. `ArrayList`):

- **è passiva rispetto alla nostra applicazione — siamo noi ad importarla, istanziarla e chiamarne i metodi quando vogliamo**
- La nostra classe è **passiva** rispetto al framework:
 - **non parte finché Spring non la invoca**
- Il framework (Spring) è la parte **attiva**:
 - **è lui che decide quando istanziare le nostre classi e quando invocare i loro metodi**

Questo è esattamente ciò che abbiamo fatto finora:

- abbiamo annotato il Controller con `@RestController` e Spring lo ha istanziato e invocato automaticamente quando arrivava una richiesta HTTP. Noi non abbiamo mai scritto `new UtenteController()`.

👁 Il principio di Hollywood:

Lo IoC viene anche chiamato "principio di Hollywood" — *"Don't call us, we'll call you"*.

Non sei tu a chiamare il framework, è il framework a chiamare te.

☰ Esempi di IoC che abbiamo già visto

`ArrayList` vs Spring:

- quando usi un `ArrayList` sei tu l'attore attivo:
 - **importi la libreria e chiami i suoi metodi.**
 - **Con Spring invece è il contrario: sei tu a diventare passivo e Spring a chiamare il tuo codice.**

I Thread:

- il metodo `run()` non va mai chiamato direttamente perché non attiverebbe il multithreading.
- Va chiamato `start()`, che è il framework della JVM a invocare `run()` nel momento giusto.
- È un esempio di IoC: non sei tu a decidere quando parte `run()`, è la JVM.

Nota pratica:

Per funzionare correttamente, le classi gestite da Spring devono trovarsi nel pacchetto base del progetto o in un suo sottopacchetto.

Se il Controller si trova fuori da questo perimetro, Spring non lo trova e non lo istanzia.

Per comprendere meglio il IoC, facciamo riferimento all'immagine:



Il protagonista di questo esempio è il componente `MyApplicationComponent` :

- Nel caso classico usato fino ad ora con java base(escludendo il multithreading) è la nostra applicazione (ad esempio la classe con il `main`) a chiamare le librerie di Java(JSE Library).
Quando dovevamo utilizzare i componenti come `ArrayList`, `HashMap`, `TreeSet`, `String`, etc. importavamo la libreria, istanziavamo esplicitamente un oggetto di queste classi e tramite dot notation usavamo i loro metodi.
Questo è l'esempio classico di controllo:

- **è la nostra applicazione ad essere la parte attiva che chiama la libreria — che rimane passiva.**

Con Spring invece:

- **è la nostra applicazione a diventare passiva, mentre il framework diventa la parte attiva che la chiama.**

Quindi nel caso di Spring invece, ma più in generale di qualsiasi framework di qualsiasi linguaggio(Flask o React), è la nostra applicazione a diventare la

parte passiva perché è il framework a chiamarla usando le sue clausole e regole.

Tornando agli esempi fatti nelle lezioni precedenti di Spring dove nella classe Controller o Service usavamo le annotazioni `@RestController` o `@Service` sopra la firma di queste due classi, notiamo che è Spring a istanziare implicitamente in modo automatico l'oggetto di queste due classi.

Non siamo noi a farlo, a differenza di quanto fatto finora nelle classi `DAO` o `Service` dove, tramite **incapsulamento**, rispettivamente istanziamo esplicitamente gli oggetti delle `Entity` o dei `DTO`.

Mentre con le annotazioni `@RestController` o `@Service` non ci è mai servito istanziare esplicitamente gli oggetti della Controller o della Service per poter utilizzare i metodi di queste due classi, questo perché:

- **queste due annotazioni dicono a spring di istanziare implicitamente queste due classi come una Controller o come una Service allo startup dell'applicazione.**

Quindi è la nostra applicazione ad aspettare che il framework la chiami, la istanzi e usi i suoi metodi.

Dependency Injection

La **Dependency Injection** è una forma specifica di IoC:

- **Consiste nella capacità di un framework — detto Container — di creare oggetti in inversione di controllo, occupandosi anche di creare e iniettare automaticamente gli oggetti di cui sono composti, detti dipendenze.**

L'esempio del Musicista

Supponiamo di avere una classe `Player` che ha una dipendenza da `Instrument` e di voler decidere il tipo di strumento **dinamicamente** — senza sapere a compile time se il `Player` suonerà una chitarra, un violino o una batteria.

Il problema è lo stesso che abbiamo visto nell'esercizio degli utenti:

- **se `Player` istanzia esplicitamente `new Guitar()`, è accoppiato a `Guitar` — non è più flessibile.**

La soluzione è:

1. **Creare un'interfaccia `Instrument` con un metodo `play()`**
2. **Creare le classi concrete `Guitar`, `Violin`, `Drum` che la implementano**

3. Chiedere a Spring di creare il `Player` e di iniettarli dinamicamente lo strumento corretto

In questo modo `Player` non sa e non gli importa quale strumento suonerà — lavora solo con l'interfaccia `Instrument`. È Spring a decidere quale implementazione concreta iniettare.

🔗 Il Collegamento con la Cartoleria:

Questo concetto lo abbiamo già visto in Java base con l'esercizio della cartoleria:

la classe `Magazzino` non nominava mai esplicitamente `Gomma` o `Penna` — lavorava con la classe padre `Articolo`.

Grazie al binding dinamico e al polimorfismo, era in grado di gestire entrambi i prodotti senza dipendere esplicitamente da nessuno dei due.

La Dependency Injection porta questo concetto al livello successivo:

- **non sei tu a creare l'oggetto concreto e passarlo alla classe che ne ha bisogno — è il Container di Spring a farlo automaticamente, leggendo le annotazioni e costruendo il grafo delle dipendenze.**

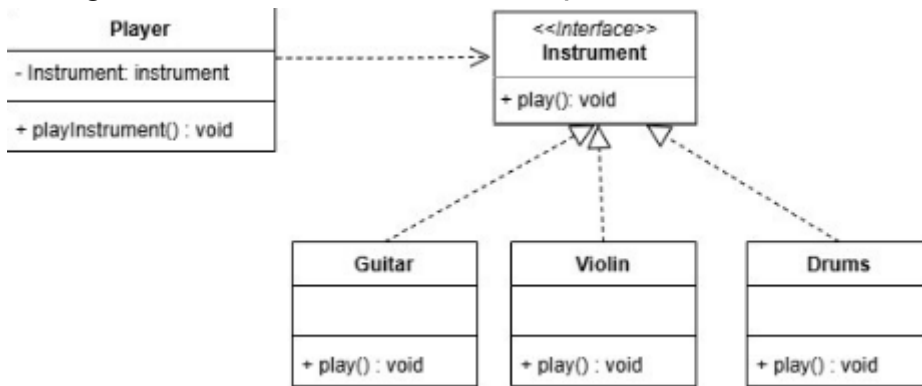
👁 In pratica:

se si danno al Controller le giuste coordinate tramite le annotazioni, Spring è in grado di creare da solo il Service, il DAO, i DTO e le Entity — senza che tu scriva mai un `new`.

📜 Ogni componente dichiara solo di cosa ha bisogno, e Spring pensa a procurarglielo.

Diagramma delle classi delle esempio player

Il diagramma delle classi dell'esempio del musicista è strutturato così:



- `Instrument` — è un'interfaccia con un metodo astratto `play()` , che verrà implementato dalle sottoclassi concrete
- `Guitar` , `Violin` , `Drum` — sono le classi concrete che implementano `Instrument` e forniscono ognuna la propria implementazione di `play()`
- `Player` — è la classe principale, che ha un attributo di tipo `Instrument` e un metodo `playInstrument()`

Il punto chiave è che `Player` dichiara l'attributo come riferimento all'**interfaccia** `Instrument` — non a una classe concreta:



Java

```
1 private Instrument instrument;
```

Questo permette di fare **chiamate polimorfiche** al metodo `play()` : a runtime, a seconda di quale oggetto concreto (`Guitar` , `Violin` o `Drum`) viene iniettato da Spring nell'attributo `instrument` , verrà eseguita l'implementazione corrispondente. `Player` non sa e non gli importa quale strumento suonerà — sa solo che l'oggetto che riceve implementa l'interfaccia `Instrument` e quindi ha un metodo `play()` .

Una volta create le classi, basterà configurarle tramite le annotazioni di Spring affinché il Container possa:

1. **Creare** l'oggetto `Player`
2. **Creare** lo strumento che vogliamo fargli suonare
3. **Iniettare** lo strumento nel `Player` automaticamente

Nota — Pattern Strategy:

Questo esempio è un'applicazione diretta del **pattern Strategy** che abbiamo visto [nella lezione sui Design Pattern](#).

`Instrument` è la strategia astratta, `Guitar`, `Violin` e `Drum` sono le strategie concrete intercambiabili, e `Player` è il contesto che le usa senza sapere quale implementazione concreta sta ricevendo.

La Dependency Injection di Spring potenzia questo pattern:

- **invece di essere tu a passare manualmente la strategia concreta al contesto, è Spring a iniettarla dinamicamente a runtime — rendendo il sistema ancora più disaccoppiato e flessibile.**

Non è da confondere con il pattern **Adapter** che abbiamo visto con JDBC — lì si trattava di far comunicare due interfacce incompatibili, qui si tratta di rendere intercambiabili diverse implementazioni della stessa interfaccia.

@Component e @Autowired

Spring, al fine di implementare la IoC e la Dependency Injection, usa le cosiddette **annotazioni**. Come abbiamo già visto con `@RestController` e `@Service`, le annotation servono a Spring per due scopi principali:

1. **Indicare il ruolo** del componente nella struttura del progetto — Controller, Service, DAO, ecc.
2. **Configurare come gestirlo** — ad esempio **quando istanziarlo, quali dipendenze iniettargli e con quale priorità sceglierlo tra più implementazioni disponibili.**

In tutti i casi il principio è lo stesso:

- **non sei tu a scrivere `new` — è Spring a leggere le annotation allo startup e a costruire e collegare automaticamente tutti i componenti.**

Nota — @Annotation **Spring vs Decorator Python:**

Le annotation di Spring e i [decorator Python](#) sono concettualmente simili:

entrambi usano la sintassi `@` per aggiungere comportamento a una classe senza modificarne il codice.

La differenza principale è nel funzionamento sottostante:

- **I decorator Python:**

- **wrappano la funzione originale ed eseguono codice aggiuntivo a runtime**
- Le **annotation Java** sono **metadati** che Spring **legge allo startup per capire come gestire le classi:**
 - **non modificano il comportamento della classe direttamente, ma dicono al Container come istanziarla e collegarla alle altre**

In entrambi i casi però l'obiettivo è lo stesso:

configurare il comportamento senza toccare il codice della classe.

Ovviamente non esistono solo queste due annotazioni ma c'è ne sono molte altre e le principali sono:

1. @Component

@Component è:

- **l'annotazione base che si pone su una classe per dire a Spring di istanziarla e gestirla automaticamente tramite il Container IoC.**

Una classe annotata con @Component viene detta **bean** di Spring.

Nota — Container Spring vs Docker:

Il termine "Container" in Spring non ha nulla a che fare con i container Docker. Il **Container di Spring** è il componente interno — chiamato `ApplicationContext` :

- **si occupa di creare, gestire e collegare tutti i bean annotati con @Component .**

È essenzialmente il "gestore" degli oggetti dell'applicazione.

Docker invece è un ambiente isolato per eseguire applicazioni — due concetti completamente diversi che condividono solo il nome.

Per funzionare correttamente con Spring, un Component dovrebbe essere un **JavaBean**, ovvero avere:

- **Costruttore a zero argomenti — necessario per la deserializzazione e per l'istanziamento da parte di Spring**

- **Getter e setter per le proprietà** — necessari per accedere e modificare i campi dell'oggetto

2. @Autowired

Se un Component ha una proprietà di tipo oggetto — quindi non primitiva (`int` , `double` , ecc.) né `String` — **è possibile chiedere a Spring di crearla e iniettarla automaticamente usando l'annotazione `@Autowired` sulla proprietà.**

Per funzionare, la classe della proprietà da iniettare deve essere a sua volta annotata con `@Component` — altrimenti Spring non sa come crearla.



Java

```
1  @Component
2  public class Player {
3
4      @Autowired
5      private Instruments inst; // Spring creerà e inietterà
    automaticamente l'implementazione corretta
6
7      public Player() { }
8
9      public void playInstruments() {
10         this.inst.play();
11     }
12 }
```

In questo esempio:

- `@Component` :
 - su `Player` dice a Spring di istanziare il `Player` come un **Component**
- `@Autowired` su `inst` dice a Spring di:
 - cercare un Component che implementi `Instruments` e di iniettarlo automaticamente nel campo `inst`

🔗 **Nota il collegamento con la Dependency Injection:**

non siamo noi a scrivere `this.inst = new Guitar()` :

- **è Spring a decidere quale implementazione concreta di Instruments iniettare.**
- **Player non sa e non gli importa quale strumento riceverà.**

3. @Primary

Ma cosa succede se esistono **più Component** che implementano la stessa interfaccia?

Ad esempio se sia `Guitar` che `Drums` sono annotate con `@Component`, Spring non sa quale delle due iniettare nel `Player` e lancerebbe un errore.

La soluzione è `@Primary`:

- **si aggiunge sotto `@Component` sulla classe che si vuole venga scelta prioritariamente rispetto alle altre.**



Java

```
1  @Component
2  @Primary
3  public class Drums implements Instruments {
4      @Override
5      public void play() {
6          System.out.println("I'm playing drum");
7      }
8  }
```

In questo modo quando Spring deve iniettare un `Instruments` nel `Player`, sceglierà `Drums` perché è marcata come primaria.

Il Problema delle Dipendenze Ambigue

Uso delle annotation

```
@Component
public class Player {

    @Autowired
    private Instrument instrument;

    public Player() {}

    public void playMyInstrument() {
        this.instrument.play();
    }
}
```

```
@Component
public class Violin implements Instrument {

    @Override
    public void play() {
        System.out.println("I'm playing the violin");
    }
}
```

```
@Component
public class Drums implements Instrument {

    @Override
    public void play() {
        System.out.println("I'm playing the drums");
    }
}
```

Se c'è più di una classe di tipo **Instrument** annotata con **@Component** Spring va in errore

Guardando l'immagine, vediamo il `Player` con `@Autowired` sull'attributo `Instrument`.

Spring deve iniettare un'implementazione concreta di `Instrument` — ma nel progetto ne esistono due: `Violin` e `Drums`, entrambe annotate con `@Component`.

Questo crea un problema:

- **Spring non sa quale delle due scegliere e va in errore.**
- È un errore di **dipendenza ambigua** — **Spring trova più candidati validi per l'iniezione e non può decidere autonomamente quale usare.**



Java

```
1 NoUniqueBeanDefinitionException: expected single matching bean
  but found 2: violin, drums
```

La soluzione che abbiamo già visto è `@Primary`:

- **si annota con `@Primary` la classe che si vuole venga scelta prioritariamente, e Spring la userà come implementazione di default quando trova più candidati dello stesso tipo.**



Java

```
1 @Component
2 @Primary
3 public class Drums implements Instruments {
```

```

4      @Override
5      public void play() {
6          System.out.println("I'm playing drum");
7      }
8  }

```

Nota il parallelismo con il pattern **Strategy** che abbiamo visto prima:

`@Primary` è il meccanismo con cui diciamo a Spring quale strategia concreta usare di default — senza che `Player` debba sapere nulla di questa scelta.

Spring Core — L'ApplicationContext

Il Spring Core è:

- **il modulo fondamentale di Spring che fornisce il Container IoC.**
È lui il vero protagonista di tutto ciò che abbiamo visto finora: è il componente responsabile di:
- **Creare i bean annotati con** `@Component`
- **Iniettare le dipendenze tramite** `@Autowired`
- **Gestire il ciclo di vita di tutti gli oggetti dell'applicazione**

Il Container IoC di Spring è implementato dalla classe

`org.springframework.context.ApplicationContext` — **è questa la classe che abbiamo chiamato "Container" fino ad ora.**

Come si usa l'ApplicationContext

Una volta create e annotate le classi, si usa l' `ApplicationContext` per ottenere i bean gestiti da Spring:



Java

```

1  ApplicationContext context = new
    AnnotationConfigApplicationContext(AppConfig.class);
2
3  // chiedo al Container di darmi il bean Player
4  // Spring lo crea, gli inietta lo strumento corretto tramite
    @Autowired e me lo restituisce
5  Player player = context.getBean(Player.class);
6
7  // chiamo il metodo - Spring ha già iniettato lo strumento
    @Primary
8  player.playInstruments(); // stampa: "I'm playing guitar"

```

Nota:

non scriviamo mai `new Player()` — chiediamo al Container di darcelo tramite `getBean()`.

È l'inversione di controllo in azione:

- non siamo noi a costruire l'oggetto e le sue dipendenze, è Spring a farlo per noi secondo le configurazioni che abbiamo dichiarato tramite le annotation.

L'ApplicationContext nel Progetto Spring Boot

In un progetto Spring Boot non è necessario creare manualmente l'ApplicationContext — lo si ottiene direttamente come valore di ritorno del metodo di avvio di Spring nella **classe starter**, quella generata automaticamente da Eclipse alla creazione del progetto:



Java

```
1  @SpringBootApplication
2  public class SpringWebServicesUtenteApplication {
3      public static void main(String[] args) {
4          // SpringApplication.run() avvia Spring e restituisce
           l'ApplicationContext
5          ApplicationContext context =
           SpringApplication.run(SpringWebServicesUtenteApplication.class,
           args);
6
7          // chiedo al Container il bean Player
8          Player player = context.getBean(Player.class);
9
10         // Spring ha già iniettato lo strumento @Primary
11         player.playInstruments();
12     }
13 }
```

Se `Guitar` è configurata come `@Primary`, Spring inietta automaticamente `Guitar` nel `Player` — e `player.playInstruments()` stamperà:



Shell

```
1 I'm playing guitar
```

Se invece si cambia `@Primary` da `Guitar` a `Drums`, senza toccare nessun'altra riga di codice, `player.playInstruments()` stamperà:



Shell

```
1 I'm playing drum
```

Questo è il valore concreto della Dependency Injection:

- **cambiare il comportamento dell'applicazione modificando solo la configurazione — senza toccare il codice della classe che usa la dipendenza.**
- `Player` non sa e non gli importa quale strumento suona — è Spring a decidere.

Tecniche di Injection

Spring offre tre modi diversi per iniettare una dipendenza tramite `@Autowired`:

1. Injection Via attributo privato:

- ==è il modo più compatto e quello che abbiamo usato negli esempi.
- **Si annota direttamente il campo con `@Autowired` :**



Java

```
1 @Autowired
2 private Instrument instrument;
```

È la forma più concisa, ma la meno consigliata:

- **l'oggetto può essere istanziato senza che la dipendenza venga impostata, rendendo difficile il testing e nascondendo le dipendenze reali della classe.**

Quindi comporta il rischio del `NullPointerException` nella finestra tra costruzione e iniezione:



Java

```
1 @Component
2 public class Player {
```

```

3     @Autowired
4     private Instrument instrument; // Spring inietta DOPO la
    costruzione
5
6     public Player() { } // in questo momento instrument è ancora
    null
7
8     public void playInstruments() {
9         this.instrument.play(); // se chiamato troppo presto →
    NullPointerException
10    }
11 }

```

2. Injection Via setter:

- **si annota il metodo setter dell'attributo con `@Autowired`.**

Spring chiama il setter dopo aver istanziato l'oggetto tramite il costruttore vuoto:



Java

```

1  @Autowired
2  public void setInstrument(Instrument instrument) {
3      this.instrument = instrument;
4  }

```

Più esplicita della precedente.

Utile quando la dipendenza è opzionale o può cambiare dopo la costruzione dell'oggetto.

Anche qui si presenta lo stesso rischio del primo modo:

- **Se Spring prima chiama il costruttore vuoto per creare l'oggetto e poi chiama il setter per iniettare la dipendenza**
- **E tra questi due momenti viene chiamato un metodo della classe il rischio è che l'oggetto esiste ma la dipendenza è ancora `null`.**

Quindi nel caso del player, se qualcuno chiama `player.playInstruments()` in quel momento, ottieni un `NullPointerException`.



Java

```

1  @Component
2  public class Player {
3      private Instrument instrument;

```

```

4
5     public Player() { } // instrument è null
6
7     @Autowired
8     public void setInstrument(Instrument instrument) {
9         this.instrument = instrument; // Spring chiama questo
        dopo il costruttore
10    }
11
12    public void playInstruments() {
13        this.instrument.play();
14    }
15 }

```

3. Via costruttore:

- **si annota il costruttore che riceve la dipendenza come parametro.**
- Spring lo chiama al momento dell'istanziazione passando automaticamente l'oggetto corretto:

java



Java

```

1  @Autowired
2  public Player(Instrument instrument) {
3      this.instrument = instrument;
4  }

```

È la tecnica **raccomandata**:

- **garantisce che l'oggetto sia sempre in uno stato valido al momento della creazione, rende le dipendenze esplicite e facilita il testing (si può istanziare la classe manualmente passando un mock).**

In questo caso invece, l'injection via costruttore invece questo problema non esiste — **la dipendenza viene iniettata nel momento stesso in cui l'oggetto viene creato, quindi non può mai esistere un Player senza uno strumento.**



Java

```

1  @Component
2  public class Player {

```



```

3     private Instrument instrument;
4
5     @Autowired
6     public Player(Instrument instrument) {
7         this.instrument = instrument; // Spring inietta DURANTE
    la costruzione
8     } // quando l'oggetto esiste, instrument è già valorizzato
9
10    public void playInstruments() {
11        this.instrument.play(); // instrument non può mai essere
    null
12    }
13 }

```

👁 **A partire da Spring 4.3, se il costruttore è unico, `@Autowired` può essere omesso.**

✍ **Le tre tecniche producono lo stesso risultato: Spring inietta la dipendenza in tutti e tre i casi.**

La differenza è **quando** avviene l'iniezione:

- **via attributo e via setter avvengono dopo l'istanziamento,**
- **via costruttore avviene durante l'istanziamento.**
- Per questo motivo l'injection via costruttore è considerata la più robusta:
 - **garantisce che l'oggetto non esista mai in uno stato in cui la dipendenza è `null`.**

Configurazione alternativa: classe `@Configuration`

Fino ad ora abbiamo usato `@Component` per dire a Spring di gestire le nostre classi.

Ma questo approccio ha un limite preciso: f

- **funziona solo su classi di cui possiedi il codice sorgente.**

Se hai bisogno di iniettare un oggetto di una libreria esterna — ad esempio un client HTTP, un oggetto di configurazione di Jackson, o qualsiasi altra classe di cui non puoi modificare il codice — non puoi aggiungere `@Component` perché non hai accesso al sorgente.

La soluzione è la classe `@Configuration` con i metodi `@Bean`:



Java

```
1  @Configuration
2  public class AppConfig {
3
4      @Bean
5      public Instrument instrument() {
6          return new Guitar(); // Spring registra Guitar come bean
           di tipo Instrument
7      }
8
9      @Bean
10     public Player player() {
11         return new Player(instrument()); // Spring crea Player
           iniettandogli il bean Instrument
12     }
13 }
```

Come funziona

- `@Configuration` :
 - **marca la classe come sorgente di definizioni di bean — Spring la legge allo startup per sapere quali oggetti creare**
- `@Bean` :
 - **si mette su un metodo il cui valore di ritorno viene registrato come bean nel Container — è l'equivalente programmatico di `@Component`**
- `@Bean @Primary` :
 - funziona come `'@Primary'` su `@Component` — **il bean viene scelto prioritariamente in caso di più candidati dello stesso tipo**

Il confronto con `@Component`

	<code>@Component</code>	<code>@Bean</code> in <code>@Configuration</code>
Dove si mette	Sulla classe	Sul metodo che crea l'oggetto
Quando si usa	Classi di cui hai il sorgente	Classi di librerie esterne
Flessibilità	Meno flessibile	Più flessibile — puoi configurare l'oggetto prima di restituirlo

❓ Quando è indispensabile

Questo approccio è **necessario** quando:

- **si vuole iniettare un oggetto di cui non si possiede il codice sorgente (es. una classe di una libreria esterna), su cui non è quindi possibile aggiungere `@Component`.**

☰ In sintesi:

`@Component` e `@Bean` sono due modi diversi per registrare un bean nel Container di Spring.

Il risultato è lo stesso: **Spring gestisce l'oggetto e lo inietta dove serve, ma `@Bean` è indispensabile quando non puoi modificare il codice della classe da iniettare.**

Esempio Pratico — `@Configuration` con la Batteria



Java

```
1  @Configuration
2  public class AppConfig {
3
4      @Bean
5      public Instrument instrument() {
6          return new Drums(); // Spring registra Drums come bean di
           tipo Instrument
7      }
8
9      @Bean
10     public Player player() {
11         return new Player(instrument()); // Spring crea Player
           iniettandogli Drums
12     }
13 }
```

I metodi hanno nome arbitrario — quello che conta non è il nome del metodo ma il **tipo restituito**:

- **Spring usa il tipo di ritorno per capire quale dipendenza soddisfare quando trova un `@Autowired` su un attributo di quel tipo.**

Con questa configurazione, quando Spring crea il `Player` e deve iniettare un `Instrument`, trova il bean `Drums` registrato tramite `@Bean` e lo inietta automaticamente.

Il risultato è un musicista che suona la batteria:



Shell

```
1 I'm playing drum
```

Esempio — Iniettare un Oggetto di Libreria Esterna

Come abbiamo detto, `@Component` **non può essere usato su classi di cui non si possiede il codice sorgente.**

Un esempio concreto è `LocalDate` della libreria standard Java — non possiamo aggiungere `@Component` su `LocalDate` perché è una classe di Oracle, non nostra.

In questo caso è **obbligatorio** usare la classe `@Configuration`:



Java

```
1 @Configuration
2 public class AppConfig {
3
4     // registriamo LocalDate.now() come bean di tipo LocalDate
5     @Bean
6     public LocalDate oggi() {
7         return LocalDate.now();
8     }
9
10    // registriamo una data specifica come bean alternativo
11    @Bean
12    @Primary
13    public LocalDate dataFissa() {
14        return LocalDate.of(2026, 1, 1);
15    }
16 }
```

Ora qualsiasi Component che ha bisogno di un `LocalDate` può riceverlo tramite `@Autowired`:



Java

```
1  @Component
2  public class Calendario {
3
4      @Autowired
5      private LocalDate data; // Spring inietta dataFissa() perché
   è @Primary
6
7      public void stampaData() {
8          System.out.println("Data: " + data);
9      }
10 }
```

Questo è il caso d'uso più importante di `@Configuration` :

- ogni volta che hai bisogno di iniettare un oggetto di una libreria esterna — un client HTTP, una connessione a un database, un oggetto di configurazione — non puoi usare `@Component` e devi necessariamente dichiararlo come `@Bean` in una classe `@Configuration` .

Dove Usare l'Injection — Interfacce e Implementazioni

Ora che abbiamo visto come funziona la Dependency Injection, è il momento di applicarla correttamente alla nostra architettura Controller/Service/DAO.

Le dipendenze nella nostra architettura sono due:

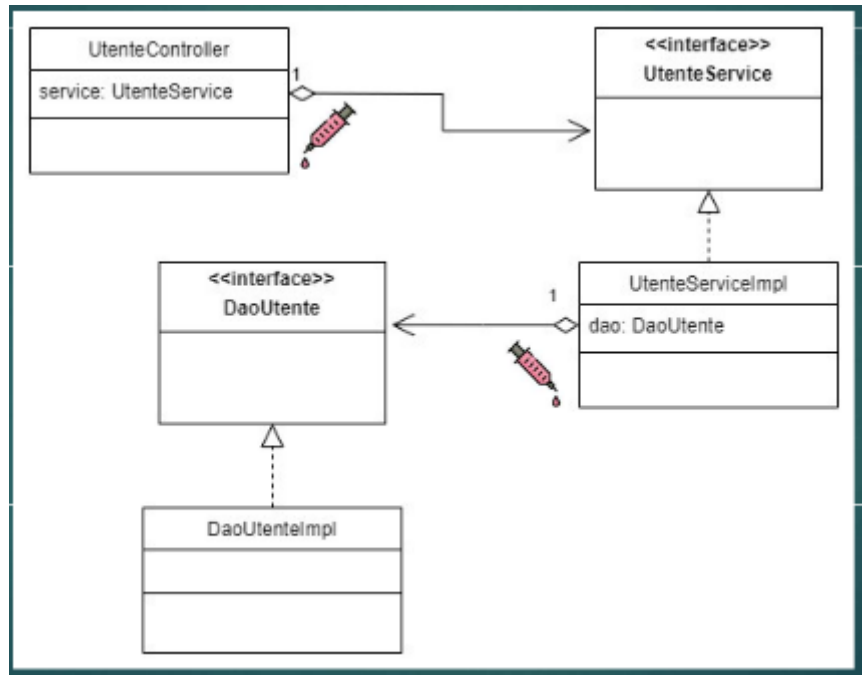
- Il **Controller** dipende dal **Service**
- Il **Service** dipende dal **DAO**

Tuttavia c'è un requisito importante: Spring per implementare l'injection usa internamente un meccanismo chiamato **proxy** — crea un oggetto intermediario che "avvolge" il bean da iniettare. Per funzionare correttamente, questo meccanismo richiede che l'oggetto da iniettare abbia sia un'**interfaccia** che un'**implementazione concreta**.

Questo significa che nella nostra architettura:

- Per ogni **Service** dovremo avere un'interfaccia e una classe che la implementa
- Per ogni **DAO** dovremo avere un'interfaccia e una classe che la implementa

Guardando il diagramma delle classi:



Screenshot 2026-03-13 at 11-44-16 Microsoft PowerPoint - Spring e Servizi REST - Spring e Servizi REST.pdf.png

A partire da sinistra:

- **UtenteController** ha un attributo `service` di tipo interfaccia **UtenteService**
 - Difatti, partecipa a un link di associazione con l'interfaccia **UtenteService**, ed ha la responsabilità su questa associazione
 - Quindi incapsula un'istanza dell'interfaccia
- **UtenteService** è l'interfaccia, ovvero il "ponte" di comunicazione tra la Controller e l'implementazione concreta dell' **UtenteService**.
 - Possiamo quindi supporre che implementi metodi astratti che andranno poi implementati concretamente nella/e classe/i concrete che implementano **UtenteService**
- **UtenteServiceImpl**: è la classe concreta che implementa l'interfaccia **UtenteService**.
 - Quindi implementerà anche i metodi astratti dell'interfaccia **UtenteService**
 - Inoltre ha un attributo `dao` di tipo interfaccia **DaoUtente**
 - Difatti partecipa a un link di associazione con l'interfaccia **DaoUtente**, ed ha una responsabilità su questa associazione verso l'interfaccia.
 - Per questo incapsula un'istanza dell'interfaccia come attributo
- **DaoUtente**: interfaccia che fa da ponte tra la service e il DAO

- Implementerà sicuramente i metodi astratti che andranno poi concretizzati nella classe concreta del DAO.
- `DAOUtenteImpl` : è la classe concreta che implementa l'interfaccia `DaoUtente`
 - Quindi implementa e specializza i metodi astratti dell'interfaccia.

Nota il parallelismo con l'esempio del musicista:

esattamente come `Player` aveva un attributo di tipo **interfaccia** `Instrument` e Spring iniettava l'implementazione concreta (`Guitar` , `Drums` , ecc.), qui il Controller ha un attributo di tipo interfaccia `UtenteService` e Spring inietta `UtenteServiceImpl` .

Il pattern è identico — cambia solo il contesto applicativo.

Perché le interfacce tra i layer?

Le interfacce tra Controller/Service e Service/DAO non sono una scelta stilistica:

- **sono il meccanismo che garantisce il disaccoppiamento tra i layer.**

Per capire perché, partiamo da un problema concreto.

Immagina di essere il programmatore che ha scritto `UtenteServiceImpl` con una dipendenza diretta su `DAOUtenteImpl` — quello che usava la `HashMap`.

Un giorno il tuo capo ti dice: "dobbiamo passare a PostgreSQL". Cosa succede?

Senza interfaccia, `UtenteServiceImpl` conosce esplicitamente `DAOUtenteImpl` — sa che è una `HashMap`, sa come è costruita, dipende dai suoi metodi specifici.

Per sostituirla con una nuova implementazione PostgreSQL dovresti:

- **Riscrivere** `DAOUtenteImpl` **da zero**
- **Modificare** `UtenteServiceImpl` **per adattarla alla nuova implementazione**
- **Rischiare di introdurre bug in un codice che funzionava già**

Con l'interfaccia invece il ragionamento cambia completamente.

`UtenteServiceImpl` non conosce `DAOUtenteImpl` :

- conosce solo il **contratto** definito dall'interfaccia `DaoUtente` .

- Quel contratto dice:
 - **"chiunque implementi DaoUtente deve avere i metodi insert(), selectAll(), selectById(), delete()".**
 - **Non dice nulla su come questi metodi funzionano internamente.**

Quindi quando arriva la richiesta di passare a PostgreSQL:

- DaoUtente (interfaccia) — **rimane invariata: il contratto non cambia**
- DAOUtenteImpl con HashMap — **viene affiancata da una nuova classe**
- DAOUtentePostgresImpl — **nuova classe che implementa la stessa interfaccia DaoUtente, rispettando lo stesso contratto**
- UtenteServiceImpl — **rimane completamente invariata:**
 - **stava parlando con il contratto, non con l'implementazione.**
 - **Non sa e non le importa se dietro c'è una HashMap o PostgreSQL.**

L'unica cosa che cambia è quale implementazione Spring inietta — **basta spostare `@Primary` da DAOUtenteImpl a DAOUtentePostgresImpl e il gioco è fatto.**

È il principio di **Single Responsibility** applicato all'architettura:

- **ogni layer ha una sola ragione per cambiare.**
 - **La Service cambia solo se cambia la logica di business.**
 - **Il DAO cambia solo se cambia il modo di accedere ai dati.**
- E grazie all'interfaccia, questi cambiamenti sono completamente **indipendenti** l'uno dall'altro.

Annotation sugli Attributi e sulle Classi

Annotation sugli Attributi

Ora che abbiamo visto il diagramma, vediamo come si traduce in codice. Per iniettare le dipendenze tra i layer si usa `@Autowired` sugli attributi di tipo interfaccia:



Java


```

1 // In UtenteController – dipende dall'interfaccia UtenteService
2 @Autowired
3 private UtenteService service; // interfaccia → Spring inietta
  UtenteServiceImpl
4
5 // In UtenteServiceImpl – dipende dall'interfaccia DaoUtente
6 @Autowired
7 private DaoUtente dao; // interfaccia → Spring inietta
  DAOUtenteImpl

```

Nota che `@Autowired` si mette sempre sull'attributo di tipo **interfaccia** — non sulla classe concreta. È Spring a decidere quale implementazione concreta iniettare.

Annotation sulle Classi — Gli Stereotipi

Finora abbiamo usato `@Component` come annotazione generica per dire a Spring di gestire una classe.

In un'applicazione Spring strutturata esistono però annotazioni più specifiche, dette **stereotipi**:

- **indicano il ruolo preciso del componente nell'architettura.**

Queste annotazioni sono **specializzazioni di `@Component`**:

- **internamente funzionano allo stesso modo — dicono a Spring di istanziare la classe e gestirla come bean — ma aggiungono un livello di semantica che `@Component` da solo non ha.**
- **In altre parole, non sono solo etichette estetiche:**
 - **comunicano esplicitamente il ruolo del componente nell'architettura, rendendo il codice autodescrittivo.**

Le tre annotazioni stereotipo che utilizzeremo sono:

1. `@RestController` :

- **si mette sulla classe concreta del `Controller`.**
- Come abbiamo già visto, non è solo un `@Component` :
 - **aggiunge anche il comportamento di serializzare automaticamente i valori di ritorno dei metodi in JSON.**
 - **È quindi uno stereotipo che combina `@Component` con la logica di presentazione REST.**

2. `@Service` :

- si mette sulla classe concreta del Service.
- Segnala a Spring che questa classe contiene logica di business.
- Oltre alla gestione come bean, Spring può applicare comportamenti specifici come la gestione delle transazioni.

3. @Repository :

- si mette sulla classe concreta del DAO.
- Segnala a Spring che questa classe si occupa di accesso ai dati.
- Spring aggiunge comportamenti specifici come:
 - la traduzione automatica delle eccezioni legate al database in eccezioni Spring standard.



Java

```

1  @RestController    // Controller - layer di presentazione
2  @RequestMapping(path="/utenti")
3  public class UtenteController { ... }
4
5  @Service          // Service - layer di logica di business
6  public class UtenteServiceImpl implements UtenteService { ... }
7
8  @Repository        // DAO - layer di accesso ai dati
9  public class DAOUtenteImpl implements DaoUtente { ... }

```

🔗 Ricapitolando:

Annotazione	Si usa su	Ruolo
@RestController	Classe concreta Controller	Layer di presentazione
@Service	Classe concreta Service	Layer di logica di business
@Repository	Classe concreta DAO	Layer di accesso ai dati

Queste tre annotazioni sono specializzazioni di `@Component`:

- funzionano allo stesso modo ma comunicano esplicitamente il ruolo del componente, rendendo il codice più leggibile e permettendo a Spring di applicare comportamenti specifici per ogni layer.

👁 Nota importante:

Le annotation si mettono SOLO sulle classi concrete — MAI sulle interfacce.

Le interfacce definiscono il contratto ma Spring non le istanzia — istanzia solo le implementazioni concrete.

Annotare un'interfaccia non avrebbe senso perché non c'è nessun oggetto da creare.

🖋 Nota pratica:

Tutte le classi annotate devono trovarsi in pacchetti che derivano dal **package base** del progetto — quello che contiene la classe di avvio con il `main`. Se una classe si trova fuori da questo perimetro, Spring non la trova durante la scansione dei componenti e non la gestisce.

Gestione degli Errori — ResponseEntity

Fino ad ora i metodi del Controller restituivano direttamente l'oggetto Java — ad esempio `UtenteDTO` o `null`.

Questo approccio ha un problema fondamentale:

- **non comunica al client lo status code HTTP dell'operazione.**

🔔 **Restituire `null` quando un utente non viene trovato non è una risposta REST appropriata:**

il client non sa se c'è stato un errore, quale tipo di errore, e perché.

Le anomalie di un web service appartengono a due famiglie principali:

1. Errori di basso livello:

- **derivanti dal tentativo di violare vincoli sul database (es. chiave duplicata, campo obbligatorio mancante)**

2. Errori applicativi:

- **derivanti da vincoli logici gestiti dalle classi Service (es. utente non trovato, operazione non permessa)**

La soluzione è `ResponseEntity<T>` : **un oggetto parametrico di Spring che modella la risposta completa per il client, contenendo :**

- Il dato di tipo `T` — **che può essere un DTO in caso di successo, o un messaggio di errore in caso di fallimento**
- Lo **status code HTTP** — **che comunica l'esito dell'operazione**



Java

```
1 // in caso di successo - restituisce l'utente con status 200 OK
2 ResponseEntity<UtenteDTO> response =
  ResponseEntity.ok(utenteDTO);
3
4 // in caso di errore - restituisce un messaggio con status 404
  Not Found
5 ResponseEntity<String> response =
  ResponseEntity.status(HttpStatus.NOT_FOUND)
6                      .body("Utente non trovato");
```

I metodi custom sono implementati dal programmatore nel Controller per trasformare gli errori interni in oggetti `ResponseEntity` appropriati — con il messaggio descrittivo e soprattutto lo status code corretto — che Spring serializzerà e invierà al client.

 **Questo rispetta il principio RESTful dell'[[Lezione 7 - Sistemi REST#Risposte auto-descrittive|autodescrittività]]:**

- **ogni risposta deve contenere tutte le informazioni necessarie per capire l'esito dell'operazione.**
NON `null`, non un oggetto vuoto — ma uno status code preciso e un messaggio che spiega cosa è successo.

Il Metodo Gestore

Il metodo gestore è:

- **il componente che intercetta le eccezioni lanciate durante l'elaborazione di una richiesta e le trasforma in oggetti `ResponseEntity` appropriati da restituire al client.**

Può essere implementato in due modi:

1. Nelle singole **Controller**:

- per gestire gli errori specifici di quella Controller.
- È utile quando errori diversi richiedono risposte diverse per ogni Controller.

2. In una classe esterna centralizzata:

- per gestire gli errori di tutte le Controller in un unico posto.
- È l'approccio preferibile perché evita la duplicazione del codice di gestione degli errori.

Struttura del Metodo Gestore



Java

```
1 // può avere nome arbitrario
2 public ResponseEntity<TipoErroreDTO> gestisciErrore(TipoEccezione
   ex) {
3     // trasforma l'eccezione in un DTO di errore
4     TipoErroreDTO errore = new TipoErroreDTO(ex.getMessage());
5     // restituisce la ResponseEntity con il DTO e lo status code
   appropriato
6     return
   ResponseEntity.status(HttpStatus.NOT_FOUND).body(errore);
7 }
```

Il metodo:

- Ha nome arbitrario
- Riceve in input il tipo di eccezione da gestire
- La trasforma in un TipoErroreDTO — un oggetto dedicato a descrivere l'errore
- Restituisce un ResponseEntity<TipoErroreDTO> con lo status code appropriato



Perché le Eccezioni Devono essere Unchecked

Per funzionare correttamente con questo meccanismo, tutti gli errori — sia di basso livello che applicativi — **devono essere modellati come eccezioni unchecked**, cioè sottotipi di `RuntimeException`.

Il motivo è pratico:

le eccezioni unchecked si propagano automaticamente lungo la catena Controller → Service → DAO senza bisogno di dichiararle nella firma di ogni metodo.

Spring le intercetta ovunque si verifichino e le passa al metodo gestore.

Con le checked invece saresti costretto a dichiarare `throws` in ogni metodo della catena — rendendo il codice verboso e difficile da mantenere.



Java

```
1 // ✅ unchecked – si propaga automaticamente senza
   dichiarazioni
2 public class UtenteNotFoundException extends
   RuntimeException {
3     public UtenteNotFoundException(String message) {
4         super(message);
5     }
6 }
7
8 // ❌ checked – obbligherebbe a dichiarare throws in ogni
   metodo della catena
9 public class UtenteNotFoundException extends Exception {
10     public UtenteNotFoundException(String message) {
11         super(message);
12     }
13 }
```

1. Metodo Gestore nella Controller

Il metodo gestore può essere implementato direttamente nella classe Controller per gestire gli errori specifici di quella Controller. Vediamo la struttura:



Java

```
1 @ExceptionHandler(RuntimeException.class)
2 public ResponseEntity<ErroreDTO> gestisciErrore(RuntimeException e
   {
3     ErroreDTO errore = new ErroreDTO(ex.getMessage());
```

```
4     return
    ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error
5 }
```

Tre elementi chiave da notare:

1. `ErroreDTO` :

- è un DTO dedicato esclusivamente a trasportare il messaggio di errore verso il client.
- Esattamente come `NomeCognomeDTO` trasportava solo nome e cognome, `ErroreDTO` trasporta solo il messaggio dell'eccezione — niente di più.

2. `@ExceptionHandler` :

- è l'annotation che dice a Spring che questo metodo deve essere invocato automaticamente quando viene lanciata un'eccezione del tipo specificato tra parentesi.
- In questo caso `RuntimeException.class` significa che:
 - il metodo gestirà tutte le eccezioni unchecked lanciate dai metodi di quella Controller.
- È l'inversione di controllo applicata alla gestione degli errori — non sei tu a chiamare il metodo gestore, è Spring a invocarlo nel momento giusto.

3. Lo status code HTTP:

- è il codice standard che comunica al client l'esito dell'operazione.
- In questo caso `INTERNAL_SERVER_ERROR` corrisponde a `500` — ma in casi più specifici si userebbe `404 NOT_FOUND` per un utente non trovato, `400 BAD_REQUEST` per una richiesta malformata, e così via.

👁 Nota:

specificando `RuntimeException.class` si gestiscono tutte le eccezioni unchecked con un unico metodo.

Se si volessero gestire errori diversi con status code diversi, si potrebbero aggiungere più metodi `@ExceptionHandler` — uno per ogni tipo di eccezione.

☰ Esempi concreti:



Java

```
1 // gestisce il caso in cui l'utente non viene trovato → 404
2 @ExceptionHandler(UtenteNotFoundException.class)
3 public ResponseEntity<ErroreDTO>
4     >>gestisciNotFound(UtenteNotFoundException ex) {
5     ErroreDTO errore = new ErroreDTO(ex.getMessage());
6     return ResponseEntity.status(HttpStatus.NOT_FOUND).body(
7     }
8 // gestisce il caso in cui si tenta di registrare un utente
9 // esistente → 409
10 @ExceptionHandler(UtenteGiaEsistenteException.class)
11 public ResponseEntity<ErroreDTO>
12     >>gestisciConflitto(UtenteGiaEsistenteException ex) {
13     ErroreDTO errore = new ErroreDTO(ex.getMessage());
14     return ResponseEntity.status(HttpStatus.CONFLICT).body(
15     }
16 // gestisce tutti gli altri errori imprevisti → 500
17 @ExceptionHandler(RuntimeException.class)
18 public ResponseEntity<ErroreDTO>
19     >>gestisciErroreGenerico(RuntimeException ex) {
20     ErroreDTO errore = new ErroreDTO(ex.getMessage());
21     return
22     ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).bo
23 }
```

Ogni metodo gestisce un tipo di eccezione specifico e restituisce lo status code semanticamente corretto:

- 404 — la risorsa non esiste
- 409 CONFLICT — conflitto con lo stato attuale della risorsa (es. duplicato)
- 500 — errore generico imprevisto, usato come "rete di sicurezza" per tutto ciò che non è stato gestito esplicitamente

2. Classe Centralizzata — @ControllerAdvice

Invece di replicare il metodo gestore in ogni Controller, **è possibile centralizzare la gestione degli errori in un'unica classe dedicata che intercetta le eccezioni di tutte le Controller del progetto** :



Java

```
1  @ControllerAdvice
2  public class GestoreErroriGlobale {
3
4      @ExceptionHandler(RuntimeException.class)
5      public ResponseEntity<ErroreDTO> gestisciErrore(RuntimeExcepti
6  ex) {
7          ErroreDTO errore = new ErroreDTO(ex.getMessage());
8          return
9      ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error
```

Tre elementi chiave:

1. `@ControllerAdvice` :

- **è l'annotation che dice a Spring che questa classe è un intercettore globale su tutte le chiamate alle RestController del progetto.**
- **Non è una Controller, non è un Service:**
 - **è un componente trasversale che si "mette in mezzo" tra le Controller e il client per catturare gli errori prima che raggiungano il client senza una risposta appropriata.**

2. `@ExceptionHandler(RuntimeException.class)` :

- **funziona esattamente come nella Controller singola, ma con portata globale:**
 - **questo metodo gestisce le eccezioni unchecked lanciate da qualsiasi Controller del progetto.**

3. `ErroreDTO` :

- **lo stesso DTO che abbiamo visto prima, trasporta il messaggio di errore verso il client.**

Confronto tra i due approcci

	Metodo nella Controller	Classe centralizzata
Portata	Solo quella Controller	Tutte le Controller
Quando usarla	Errori specifici di una Controller	Errori comuni a tutto il progetto
Duplicazione	Alta — va replicata in ogni Controller	Nulla — un unico punto di gestione

🔗 È buona pratica avere **sempre una classe** `@ControllerAdvice` **con un** `@ExceptionHandler(RuntimeException.class)` **come "rete di sicurezza" globale — e aggiungere gestori specifici nelle singole Controller solo quando necessario.**

❓ Come funziona `@ControllerAdvice` con l'loC:

La classe `@ControllerAdvice` **non si importa nella Controller né si estende — è un componente indipendente che Spring gestisce automaticamente tramite loC.**

Non c'è nessuna relazione esplicita tra la Controller e la classe centralizzata nel codice.

Il flusso è il seguente:

1. **Spring trova la classe** `@ControllerAdvice` **durante la scansione dei componenti allo startup**
2. **La registra automaticamente come intercettore globale per tutte le Controller del progetto**
3. **Quando una Controller lancia un'eccezione, Spring la intercetta e la passa al metodo** `@ExceptionHandler` **corretto nella classe centralizzata**
4. **Se una singola Controller ha un proprio** `@ExceptionHandler` **per un tipo di eccezione specifico, Spring darà priorità al gestore più specifico — quello nella Controller singola — rispetto a quello globale**

Non si importa, non si estende — Spring fa tutto automaticamente.

È l'loC applicato alla gestione degli errori.

Esempio Pratico — Gestione dei Prodotti con ResponseEntity

Questo esercizio riprende la stessa architettura dell'esercizio degli utenti, ma introduce due novità fondamentali:

- **l'uso corretto delle interfacce per `Service` e `DAO` con la `Dependency Injection`, e la gestione degli errori tramite `ResponseEntity` e `@ExceptionHandler`.**

1. Entity — Prodotto

Come nell'esercizio precedente:

- **L'Entity rappresenta la tabella nel database — ogni oggetto corrisponde a una riga.**

I due costruttori hanno gli stessi ruoli:

1. **il costruttore vuoto per la deserializzazione di Spring,**
2. **il costruttore completo per l'uso applicativo.**



Java

```
1 public class Prodotto {
2     private int id;
3     private String descrizione;
4
5     public Prodotto() { }
6
7     public Prodotto(int id, String descrizione) {
8         this.id = id;
9         this.descrizione = descrizione;
10    }
11    // getter e setter...
12 }
```

2. DTO — ProdottoDTO e ErrorDTO

In questo esercizio abbiamo due DTO con ruoli molto diversi:

1. **ProdottoDTO :**
 - **trasporta i dati del prodotto verso il client, esattamente come UtenteDTO nell'esercizio precedente.**



Java

```
1 public class ProdottoDTO {
2     private int id;
3     private String descrizione;
4     public ProdottoDTO() { }
5     public ProdottoDTO(int id, String descrizione) { ... }
6     // getter e setter... }
7 }
```

2. ErrorDTO :

- **è il DTO dedicato agli errori.**
- **Trasporta solo il messaggio di errore verso il client quando qualcosa va storto.**
- **È la risposta di tipo T che Spring serializza in JSON e inserisce nel body della ResponseEntity in caso di eccezione.**



Java

```
1 package com.spring.prodotti.dto;
2
3 public class ErrorDTO {
4     private String msg;
5     public ErrorDTO() {} //costruttore a 0 args
6     public ErrorDTO(String msg) {
7         this.msg = msg;
8     }
9
10    public String getMsg() {
11        return msg;
12    }
13 }
```

3. Mapper — Mapper

Stesso pattern dell'esercizio precedente:

- **due metodi statici di conversione tra DTO ed Entity nelle due direzioni.**



Java

```
1 package com.spring.prodotti.mapper;
2
```

```

3  import com.spring.prodotti.dto.ProdottoDTO;
4  import com.spring.prodotti.entity.Prodotto;
5
6  public class Mapper {
7
8      public static Prodotto daDTOaEntity(ProdottoDTO dto) {
9          return new Prodotto(dto.getId(), dto.getDescrizione());
10     }
11
12
13     public static ProdottoDTO daEntityaDTO(Prodotto p) {
14         return new ProdottoDTO(p.getId(), p.getDescrizione());
15     }
16
17 }
18

```

4. DAO — DAOProdotti e DAOProdottiImpl



Java

```

1  // interfaccia - definisce il contratto
2  public interface DAOProdotti {
3      public void insert(Prodotto p);
4      public Prodotto selectById(int id);
5  }

```



Java

```

1  package com.spring.prodotti.dao;
2
3  import java.util.HashMap;
4  import java.util.Map;
5
6  import javax.management.openmbean.KeyAlreadyExistsException;
7
8  import org.springframework.stereotype.Repository;
9
10 import com.spring.prodotti.entity.Prodotto;
11 // implementazione concreta dell'interfaccia
12 @Repository
13 public class DAOProdottiImpl implements DAOProdotti {
14

```

```

15     private Map<Integer, Prodotto> mappa = new HashMap<Integer,
16         Prodotto>();
17
18     @Override
19     public void insert(Prodotto p) {
20         if (mappa.containsKey(p.getId())) {
21             throw new KeyAlreadyExistsException("La chiave: " +
22                 p.getId() + " già esiste nella mappa");
23         }
24         mappa.put(p.getId(), p);
25         System.out.println("Il prodotto: " + p.getId() + " , " +
26             p.getDescrizione() + " è stato inserito nella mappa");
27     }
28
29     @Override
30     public Prodotto selectById(int id) {
31         Prodotto p = mappa.get(id);
32         if(p == null) {
33             throw new RuntimeException("id inesistente");
34         }
35         return p;
36     }

```

Rispetto all'esercizio degli utenti ci sono due differenze importanti:

1. L'interfaccia DAOProdotti :

- **il DAO è ora separato in interfaccia e implementazione.**
- **Come abbiamo visto, questo è il modo corretto per permettere la Dependency Injection e garantire il disaccoppiamento tra i layer.**

2. La gestione degli errori nel DAO:

- invece di restituire `null` quando qualcosa va storto, il DAO lancia **eccezioni unchecked** che si propagano automaticamente fino al `@ExceptionHandler` della Controller:
 - `KeyAlreadyExistsException` — **eccezione specifica per chiavi duplicate, che potrebbe essere gestita con `409 CONFLICT`**
 - `RuntimeException` — **eccezione generica per id inesistente, gestita con `500 INTERNAL_SERVER_ERROR`**

👁 Nota:

in un'applicazione reale si userebbero eccezioni custom specifiche per ogni tipo di errore — ad esempio `ProdottoNotFoundException` per l'id inesistente — in modo da poter restituire lo status code semanticamente corretto (`404 NOT_FOUND`) invece del generico `500` .

5. Service — `ProdottoService` e `ProdottoServiceImpl`



Java

```
1 package com.spring.prodotti.service;
2 import com.spring.prodotti.dto.ProdottoDTO;
3 // interfaccia - definisce il contratto
4 public interface ProdottoService {
5     public void carica(ProdottoDTO dto);
6     public ProdottoDTO cercaPerId(int id);
7 }
```



Java

```
1 // implementazione concreta
2 @Service
3 public class ProdottoServiceImpl implements ProdottoService {
4     @Autowired
5     private DAOProdotti dao; // Spring inietta DAOProdottiImpl
6     @Override
7     public void carica(ProdottoDTO dto) {
8         Prodotto p = Mapper.daDT0aEntity(dto); dao.insert(p);
9     }
10    @Override
11    public ProdottoDTO cercaPerId(int id) {
12        Prodotto trovato = dao.selectById(id);
13        return Mapper.daEntityaDT0(trovato);
14    }
15 }
```

Due cose da notare rispetto all'esercizio degli utenti:

1. `@Autowired` su `DAOProdotti` :
 - **non si usa più** `new DAOProdottiImpl()` **ma si dichiara un attributo di tipo interfaccia** `DAOProdotti` **annotato con** `@Autowired`.

- Spring inietta automaticamente DAOProdottiImpl perché è l'unica classe annotata con @Repository che implementa quella interfaccia.

2. Nessuna gestione degli errori nel Service:

- il Service non gestisce le eccezioni lanciate dal DAO.
- Le lascia propagare automaticamente verso la Controller, dove verranno catturate dal @ExceptionHandler.
- Questo è corretto — non è compito del Service gestire gli errori di presentazione.

6. Controller — ProdottoController



Java

```
1  @RestController
2  @RequestMapping(path="/prodotti")
3  public class ProdottoController {
4
5      @Autowired
6      private ProdottoService service; // Spring inietta
        ProdottoServiceImpl
7
8      @PostMapping(path="/carica", consumes="application/json")
9      public void carica(@RequestBody ProdottoDTO dto) {
10         service.carica(dto);
11     }
12
13     @GetMapping(path="/cercaId/{id}",
        produces="application/json")
14     public ProdottoDTO cercaPerId(@PathVariable int id) {
15         return service.cercaPerId(id);
16     }
17
18     // gestione degli errori - moralmente è come un catch
19     @ExceptionHandler
20     public ResponseEntity<ErrorDTO>
        gestireErrore(RuntimeException re) {
21         ErrorDTO dto = new ErrorDTO(re.getMessage());
22         return new ResponseEntity<>(dto,
            HttpStatus.INTERNAL_SERVER_ERROR);
23     }
24 }
```


Anche qui `@Autowired` su `ProdottoService` :

- Spring inietta `ProdottoServiceImpl` automaticamente.
- Il Controller non sa e non gli importa quale implementazione concreta sta usando.

Il metodo `gestoreErrore()` è il cuore della novità di questo esercizio.

Come dice il commento nel codice:

- **è moralmente come un `catch` — riceve l'eccezione come parametro esattamente come un blocco `catch` , la trasforma in un `ErrorDTO` e la restituisce al client dentro una `ResponseEntity` con lo status code `500` `INTERNAL_SERVER_ERROR` .**

La differenza rispetto a un `try-catch` classico è che non devi scriverlo in ogni metodo:

- **`@ExceptionHandler` lo applica automaticamente a tutti i metodi della Controller. È l'IoC applicato alla gestione degli errori.**

Nota sul costruttore di `ResponseEntity` :

In questo esercizio si usa `new ResponseEntity<>(dto, HttpStatus.INTERNAL_SERVER_ERROR)` invece di `ResponseEntity.status(...).body(...)` .

Sono due modi equivalenti per costruire la stessa risposta — il secondo è più leggibile, il primo è più esplicito.